

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 7

по курсу «Алгоритмы компьютерной графики»

Студент группы ИУ9-42Б Лавров Р. Д.

Преподаватель Цалкович П. А.

Москва 2026

1 Задача

- цель работы: изучение эффективных приемов организации приложений и оптимизации вызовов OpenGL.
- задача: оптимизация приложения OpenGL, созданного в рамках предыдущей лабораторной работы, на основе выбора наиболее эффективных методик. (см. Баяковский Ю.М., Игнатенко А.В. Начальный курс OpenGL.- М.: «Планета Знаний», 2007.- 221с. – Глава 9).
- обязательно использовать дисплейные списки и массивы вершин и еще 2 любые различные оптимизации(в сумме минимум 4 оптимизации).
- оценка применимости выбранного метода оптимизации приложения OpenGL должна осуществляться на основании измерения производительности.
- результаты замеров оформить в табличном виде.

2 Основная теория

В рамках лабораторной работы были рассмотрены и применены четыре метода оптимизации приложений OpenGL, направленные на снижение нагрузки на графический конвейер и уменьшение времени отрисовки кадра.

2.1 Массивы вершин (Vertex Arrays)

Вместо последовательной передачи каждой вершины через команды glVertex, glNormal, glTexCoord внутри блока glBegin/glEnd используется предварительное заполнение массивов данных. Для этого определены функции:

- glVertexPointer — задание массива координат вершин;
- glNormalPointer — массив нормалей;
- glTexCoordPointer — массив текстурных координат.

Перед использованием массивы активируются вызовами glEnableClientState. Отрисовка выполняется одной командой glDrawArrays или glDrawElements. Это позволяет сократить число вызовов OpenGL и передавать данные блоками, что повышает эффективность работы графической подсистемы.

2.2 Дисплейные списки (Display Lists)

Дисплейный список — это скомпилированный набор команд OpenGL, который сохраняется в памяти графического сервера. Создание списка производится между вызовами glNewList и glEndList. В режиме GL_COMPILE команды только записываются, но не выполняются. В дальнейшем для отрисовки достаточно одного вызова glCallList. Такая оптимизация особенно эффективна для статической геометрии, которая многократно повторяется в каждом кадре, так как драйверу не нужно заново интерпретировать и передавать тысячи примитивов.

2.3 Управление обработкой вершин в графическом конвейере

Данный подход включает несколько рекомендаций:

- Использование **направленных источников света** — координата *w* в четвёртой компоненте позиции источника устанавливается в 0 (вместо 1). Это исключает пересчёт направления света для каждой вершины.
- Отключение **двухстороннего освещения** (GL_LIGHT_MODEL_TWO_SIDE = GL_FALSE) и **локальной модели освещения** (GL_LIGHT_MODEL_LOCAL_VIEWER = GL_FALSE) — уменьшает количество вычислений при расчёте освещения.
- **Отключение нормализации** нормалей (glDisable(GL_NORMALIZE)), если все нормали заранее приведены к единичной длине. Это исключает дорогостоящую операцию нормализации для каждого вектора.
- Использование **связанных примитивов** (GL_QUAD_STRIP, GL_TRIANGLE_STRIP) для уменьшения числа передаваемых вершин.

В представленной работе нормали сферы вычисляются как $\frac{(x,y,z)}{R}$, что гарантирует их единичную длину, поэтому вызов $\mathrm{glEnable(GL_NORMALIZE)}$ был удалён. Источник света сделан направленным заменой четвёртой координаты с 1 на 0.

2.4 Управление растеризацией

Растеризация — этап преобразования геометрических примитивов в пиксели. Для ускорения этого этапа применяются:

- **Отсечение обратных граней** ($\mathrm{glEnable(GL_CULL_FACE)}$) — полигоны, повёрнутые от наблюдателя, удаляются до стадии растеризации. Для сферы это позволяет не обрабатывать около половины всех фрагментов.
- **Минимизация лишних пиксельных операций** — например, отключение альфа-смешивания, теста трафарета и т.п., если они не нужны.
- При необходимости — уменьшение разрешения окна или области вывода.

В данной работе было включено отсечение задних граней ($\mathrm{glCullFace(GL_BACK)}$) с указанием фронтального порядка вершин против часовой стрелки ($\mathrm{glFrontFace(GL_CCW)}$).

2.5 Методика измерения производительности

Для объективной оценки эффективности каждой оптимизации использовался высокоточный таймер. На операционной системе macOS применялась функция $\mathrm{clock_gettime_nsec_np(CLOCK_UPTIME_RAW)}$ или, как запасной вариант, $\mathrm{mach_absolute_time}$ с преобразованием через $\mathrm{mach_timebase_info}$. Измерялось время выполнения блока отрисовки сферы (без учёта анимации и очистки буферов). Для снижения погрешности проводилось несколько замеров:

- Первая отрисовка — время компиляции дисплейного списка или первой генерации данных.
- Среднее время последующих 10 отрисовок — оценка установившейся производительности.

3 Результаты

Результаты измерений:

Оптимизация	Первая отрисовка	Следующие 10 отрисовок (среднее)
-	1.75 мс	0.616 мс
списки вершин	0.221 мс	0.0039 мс
дисплейные списки	1.35 мс	0.051 мс
упр. обр. вершин	1.57 мс	0.608 мс
упр. растеризацией	1.54 мс	0.554 мс
всё вместе	0.658 мс	0.0485 мс

Листинг 1: Код со всеми оптимизациями

```
import glfw
from PIL import Image
from OpenGL.GL import *
import math
import time
import ctypes
import ctypes.util

class PerformanceTimer:
    def __init__(self):
        self.libc = ctypes.CDLL(ctypes.util.find_library('c'))
        try:
            self._get_time_ns = self.libc.clock_gettime_nsec_np
            self._get_time_ns.argtypes = [ctypes.c_int]
            self._get_time_ns.restype = ctypes.c_uint64
            self.use_mach = False
        except AttributeError:
            self.libkernel = ctypes.CDLL(ctypes.util.find_library('kernel'))
            self._mach_absolute_time = self.libkernel.mach_absolute_time
            self._mach_absolute_time.argtypes = []
            self._mach_absolute_time.restype = ctypes.c_uint64
            class mach_timebase_info(ctypes.Structure):
                _fields_ = [("numer", ctypes.c_uint32), ("denom", ctypes.c_uint32)]
            self._timebase = mach_timebase_info()
            self.libkernel.mach_timebase_info(ctypes.byref(self._timebase))
            self.use_mach = True

    def get_time_ns(self):
        if self.use_mach:
```

```
                ticks = self._mach_absolute_time()
                return (ticks * self._timebase.numer) // self._timebase.denom
            else:
                return self._get_time_ns(0)

angle_x, angle_y = 0, 0
wireframe = False
scale = 0.8
texture_enabled = True
texture_id = None

pos = [-0.3,  0.0,  0.0]
vel = [0.5,  0.4,  0.3]
BOUND = 1.1
SPHERE_RADIUS = 1.0
last_time = time.time()
timer = PerformanceTimer()
sphere_display_list = None


def three_point_projection():
    return [
        0.87,  -0.09,  0.98,  0.49,
        0,      0.98,  0.35,  0.17,
        0.5,    0.15,-1.7,   -0.85,
        0,      0,     1,     2
    ]

def setup_lighting():
    glEnable(GL_LIGHTING)
    glEnable(GL_LIGHT0)

    glLightfv(GL_LIGHT0, GL_AMBIENT, [0.3, 0.3, 0.3, 1])
    glLightfv(GL_LIGHT0, GL_DIFFUSE, [3, 3, 3, 1])
    glLightfv(GL_LIGHT0, GL_SPECULAR, [3, 3, 3, 1])
    glLightfv(GL_LIGHT0, GL_POSITION, [0, 3, 0, 0])

    glLightf(GL_LIGHT0, GL_SPOT_CUTOFF, 70)
    glLightfv(GL_LIGHT0, GL_SPOT_DIRECTION, [0, -1, 0])
    glLightf(GL_LIGHT0, GL_SPOT_EXPONENT, 70)

    glLightModelfv(GL_LIGHT_MODEL_AMBIENT,  [0.3, 0.3, 0.3, 1.0])
    glLightModeli(GL_LIGHT_MODEL_LOCAL_VIEWER, GL_FALSE)
    glLightModeli(GL_LIGHT_MODEL_TWO_SIDE, GL_FALSE)

def setup_material():
```

```
    glEnable(GL_COLOR_MATERIAL)
    glColorMaterial(GL_FRONT, GL_AMBIENT_AND_DIFFUSE)
    glMaterialfv(GL_FRONT, GL_SPECULAR, [0.8, 0.8, 0.8, 1.0])
    glMaterialf(GL_FRONT, GL_SHININESS, 64.0)

def create_checkerboard_texture():
    img = Image.open('3.jpg').resize((128, 128))
    img_data = img.tobytes()
    width, height = img.size
    texture = glGenTextures(1)
    glBindTexture(GL_TEXTURE_2D, texture)
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height, 0, GL_RGB,
    GL_UNSIGNED_BYTE, img_data)
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR)
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR)
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT)
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT)
    return texture

def generate_sphere_vertex_arrays(segments=10, sides=20, R=SPHERE_RADIUS):
    slices = []
    for i in range(segments):
        theta1 = -math.pi/2 + (i / segments) * math.pi
        theta2 = -math.pi/2 + ((i + 1) / segments) * math.pi
        y1 = R * math.sin(theta1)
        y2 = R * math.sin(theta2)
        r1 = R * math.cos(theta1)
        r2 = R * math.cos(theta2)
        height = y2 - y1
        if r1 >= 0 and r2 >= 0:
            vertices = []
            normals = []
            texcoords = []
            angle_step = 2 * math.pi / sides
            angle = 0.0
            for _ in range(sides + 1):
                x1 = r1 * math.cos(angle)
                z1 = r1 * math.sin(angle)
                x2 = r2 * math.cos(angle)
                z2 = r2 * math.sin(angle)
                y_top = y1 + height

                if r1 > 0.001:
                    nx1, ny1, nz1 = x1/R, y1/R, z1/R
                else:
                    nx1, ny1, nz1 = 0.0, -1.0, 0.0
```

```
                    if r2 > 0.001:
                        nx2, ny2, nz2 = x2/R, y_top/R, z2/R
                    else:
                        nx2, ny2, nz2 = 0.0, 1.0, 0.0

                    u = angle / (2 * math.pi)
                    v1 = (y1 / R + 1.0) / 2.0
                    v2 = (y_top / R + 1.0) / 2.0

                    vertices.extend([x1, y1, z1])
                    normals.extend([nx1, ny1, nz1])
                    texcoords.extend([u, v1])

                    vertices.extend([x2, y_top, z2])
                    normals.extend([nx2, ny2, nz2])
                    texcoords.extend([u, v2])

                    angle += angle_step

                vertex_count = len(vertices) // 3
                vert_arr = (ctypes.c_float * len(vertices))(*vertices)
                norm_arr = (ctypes.c_float * len(normals))(*normals)
                tex_arr = (ctypes.c_float * len(texcoords))(*texcoords)
                slices.append((vert_arr, norm_arr, tex_arr, vertex_count))
    return slices

def init_display_list():
    global sphere_display_list
    slices = generate_sphere_vertex_arrays()
    sphere_display_list = glGenLists(1)
    glNewList(sphere_display_list, GL_COMPILE)
    glEnableClientState(GL_VERTEX_ARRAY)
    glEnableClientState(GL_NORMAL_ARRAY)
    glEnableClientState(GL_TEXTURE_COORD_ARRAY)
    for vert_arr, norm_arr, tex_arr, vcount in slices:
        glVertexPointer(3, GL_FLOAT, 0, vert_arr)
        glNormalPointer(GL_FLOAT, 0, norm_arr)
        glTexCoordPointer(2, GL_FLOAT, 0, tex_arr)
        glDrawArrays(GL_TRIANGLE_STRIP, 0, vcount)
    glDisableClientState(GL_VERTEX_ARRAY)
    glDisableClientState(GL_NORMAL_ARRAY)
    glDisableClientState(GL_TEXTURE_COORD_ARRAY)
    glEndList()

def display():
    global angle_x, angle_y, scale, pos, vel, last_time, texture_enabled, texture_id
```

```
    global sphere_display_list

    current_time = time.time()
    dt = current_time - last_time
    last_time = current_time
    if dt > 0.1:
        dt = 0.016
    for i in range(3):
        pos[i] += vel[i] * dt
        if pos[i] > BOUND:
            pos[i] = BOUND
            vel[i] = -vel[i]
        elif pos[i] < -BOUND:
            pos[i] = -BOUND
            vel[i] = -vel[i]

    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

    glMatrixMode(GL_PROJECTION)
    glLoadIdentity()
    glMultMatrixf(three_point_projection())
    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()

    if texture_enabled:
        glEnable(GL_TEXTURE_2D)
        glBindTexture(GL_TEXTURE_2D, texture_id)
        glTexEnvi(GL_TEXTURE_ENV, GL_TEXTURE_ENV_MODE, GL_MODULATE)
    else:
        glDisable(GL_TEXTURE_2D)

    glPolygonMode(GL_FRONT_AND_BACK, GL_LINE if wireframe else GL_FILL)

    glTranslatef(pos[0], pos[1], pos[2])
    glScalef(scale, scale, scale)
    glRotatef(angle_x, 1, 0, 0)
    glRotatef(angle_y, 0, 1, 0)
    glColor3f(1, 1, 1)

    start = timer.get_time_ns()
    if sphere_display_list is not None:
        glCallList(sphere_display_list)
    end = timer.get_time_ns()
    print(f"Frame time: {(end - start) / 1e6:.5f} ms")

    glDisable(GL_TEXTURE_2D)
```

```
213
214 def key_callback(window, key, scancode, action, mods):
215     global angle_x, angle_y, wireframe, scale, texture_enabled
216     if action in [glfw.PRESS, glfw.REPEAT]:
217         if key == glfw.KEY_RIGHT:
218             angle_y -= 3
219         if key == glfw.KEY_LEFT:
220             angle_y += 3
221         if key == glfw.KEY_UP:
222             angle_x -= 3
223         if key == glfw.KEY_DOWN:
224             angle_x += 3
225         if key == glfw.KEY_KP_ADD or key == glfw.KEY_EQUAL:
226             scale += 0.1
227         if key == glfw.KEY_KP_SUBTRACT or key == glfw.KEY_MINUS:
228             scale = max(0.1, scale - 0.1)
229         if key == glfw.KEY_M:
230             wireframe = not wireframe
231         if key == glfw.KEY_T:
232             texture_enabled = not texture_enabled
233
234 def main():
235     global texture_id, last_time, sphere_display_list
236
237     if not glfw.init():
238         return
239     window = glfw.create_window(800, 800, "lab7", None, None)
240     if not window:
241         glfw.terminate()
242         return
243     glfw.make_context_current(window)
244     glfw.set_key_callback(window, key_callback)
245
246     glEnable(GL_DEPTH_TEST)
247     glClearColor(0.15, 0.15, 0.15, 1.0)
248
249     glEnable(GL_CULL_FACE)
250     glCullFace(GL_BACK)
251     glFrontFace(GL_CCW)
252
253     setup_lighting()
254     setup_material()
255     texture_id = create_checkerboard_texture()
256
257     init_display_list()
258
```

```
    last_time = time.time()
    while not glfw.window_should_close(window):
        display()
        glfw.swap_buffers(window)
        glfw.poll_events()

    if sphere_display_list:
        glDeleteLists(sphere_display_list, 1)
    glfw.destroy_window(window)
    glfw.terminate()

if __name__ == "__main__":
    main()
```

4 Вывод

На основе проведённых измерений установлено, что наилучшую производительность обеспечивает использование **массивов вершин** (vertex arrays): время последующих отрисовок составляет 0,0039 мс, что в 158 раз быстрее неоптимизированной версии и более чем в 12 раз быстрее комбинации всех методов. Дисплейные списки и оптимизации освещения/растеризации также дают положительный эффект, однако в данном приложении наиболее рациональным является применение только массивов вершин.